1. Implement a hash table using Linear Probing

code :

```
 #include <iostream>
using namespace std;

#define SIZE 10

class HashTable {
   int table[SIZE];

public:
   HashTable() {
      for (int i = 0; i < SIZE; i++)
         table[i] = -1;
   }

   void insert(int key) {
      int index = key % SIZE;

      for (int i = 0; i < SIZE; i++) {
         int newIndex = (index + i) % SIZE;

         if (table[newIndex] == -1) {
            table[newIndex] = key;
            cout << key << " inserted at index "
                 << newIndex << endl;
            return;
         }
      }

      cout << "Hash table is full!" << endl;
   }

   void display() {
```

```
        cout << "\nHash Table:\n";

        for (int i = 0; i < SIZE; i++) {
            cout << i << " : ";

            if (table[i] == -1)
                cout << "Empty";
            else
                cout << table[i];

            cout << endl;
        }
    }
};

int main() {
    HashTable h;

    int n, key;

    cout << "Enter number of elements: ";
    cin >> n;

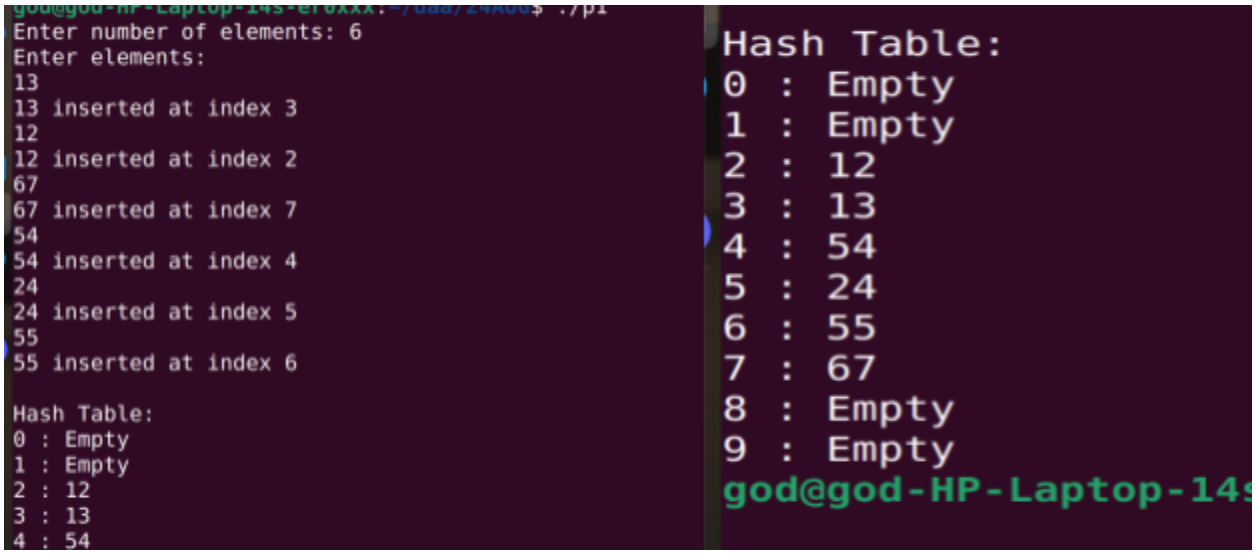
    cout << "Enter elements:\n";

    for (int i = 0; i < n; i++) {
        cin >> key;
        h.insert(key);
    }

    h.display();

    return 0;
}
```

OUTPUT:



2. Implement a hash table using Quadratic Probing

CODE:

```
#include <iostream>
using namespace std;

#define SIZE 10

class HashTable {
   int table[SIZE];

public:
   HashTable() {
      for (int i = 0; i < SIZE; i++)
         table[i] = -1;
   }
   void insert(int key) {
      int index = key % SIZE;

      for (int i = 0; i < SIZE; i++) {
```

```
            int newIndex = (index + i * i) % SIZE;

            if (table[newIndex] == -1) {
                table[newIndex] = key;
                cout << key << " inserted at index "
                     << newIndex << endl;
                return;
            }
        }

        cout << "Hash table is full!" << endl;
    }
    void display() {
        cout << "\nHash Table:\n";

        for (int i = 0; i < SIZE; i++) {
            cout << i << " : ";

            if (table[i] == -1)
                cout << "Empty";
            else
                cout << table[i];

            cout << endl;
        }
    }
};

int main() {
    HashTable h;

    int n, key;

    cout << "Enter number of elements: ";
    cin >> n;
```

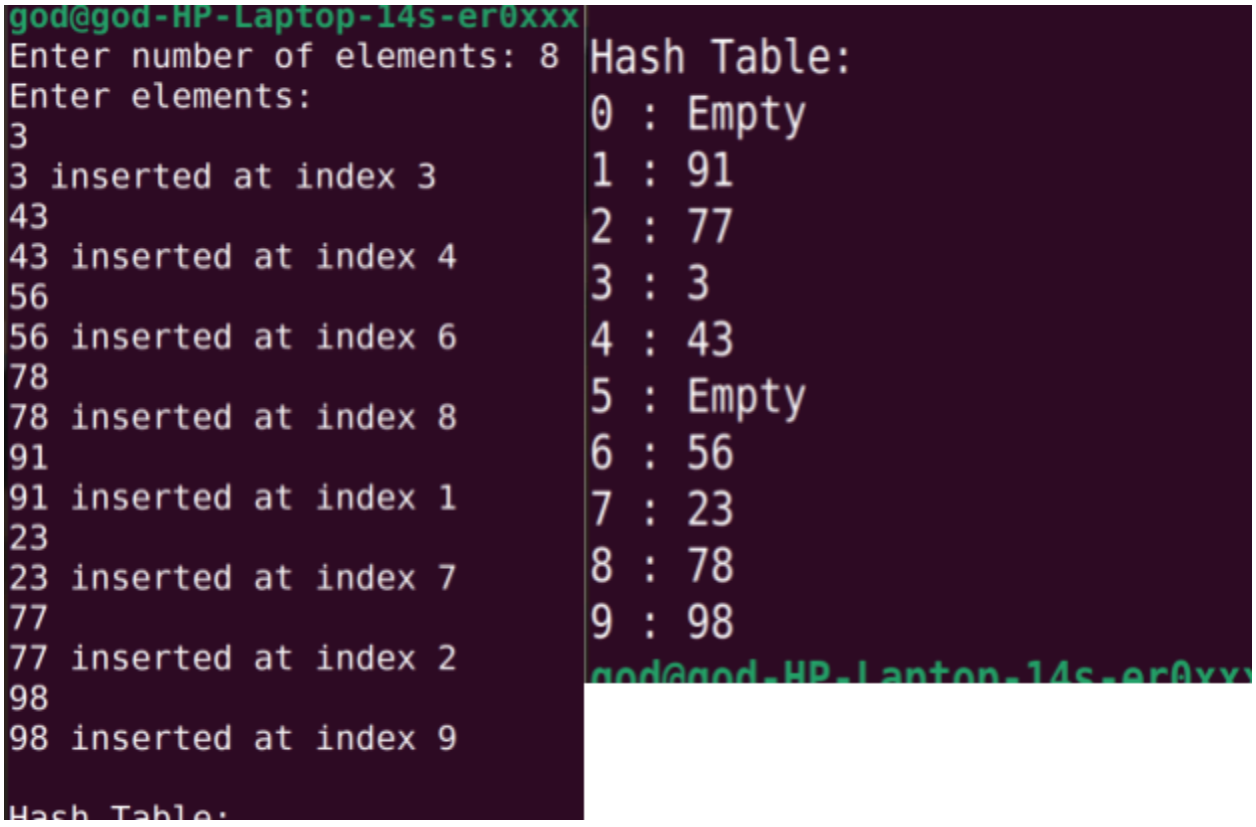
```
    cout << "Enter elements:\n";

    for (int i = 0; i < n; i++) {
        cin >> key;
        h.insert(key);
    }

    h.display();

    return 0;
}
```

OUTPUT:



3. Given an array and a target value T, determine whether there are two elements whose sum is
equal to T. (Average Time Complexity should be O(n))

CODE:

```
#include <iostream>
#include <unordered_set>
using namespace std;

int main() {
    int n, T;

    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];

    cout << "Enter array elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    cout << "Enter target value: ";
    cin >> T;

    unordered_set<int> s;
    bool found = false;

    for (int i = 0; i < n; i++) {
        int complement = T - arr[i];

        if (s.find(complement) != s.end()) {
            cout << "Pair found: " << complement
                 << " + " << arr[i]
```

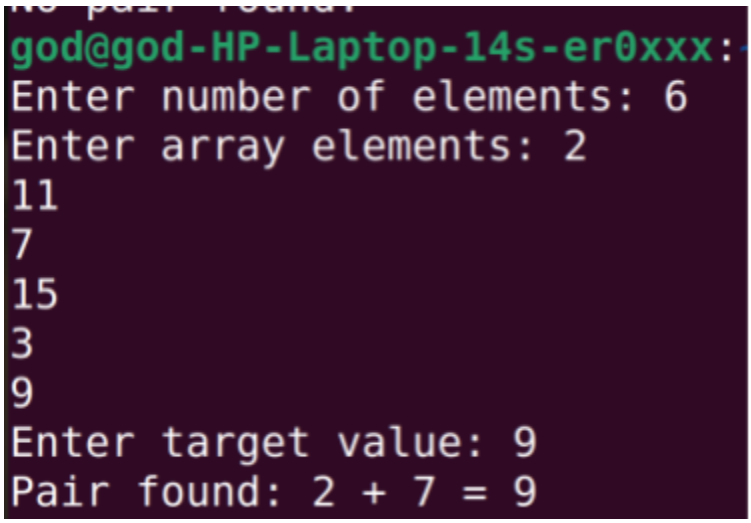
```
            << " = " << T << endl;
            found = true;
            break;
        }

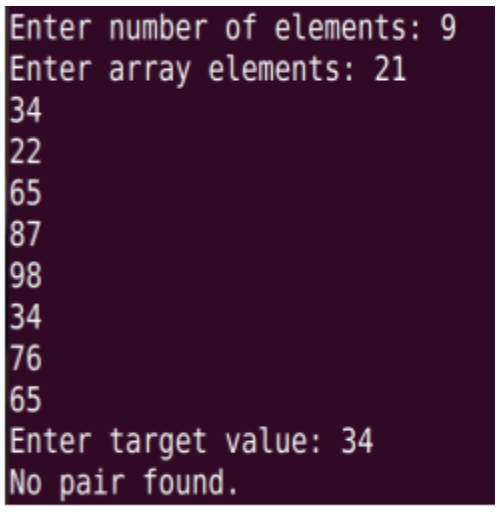
        s.insert(arr[i]);
    }

    if (!found) {
        cout << "No pair found." << endl;
    }

    return 0;
}
```

OUTPUT :





4. Given an integer array, use hashing to find the frequency of every distinct element.

CODE :

```
#include <iostream>
#include <unordered_map>
using namespace std;
```

```
int main() {
    int n;

    cout << "Enter size of array: ";
    cin >> n;

    int arr[n];

    cout << "Enter array elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    unordered_map<int, int> hash;

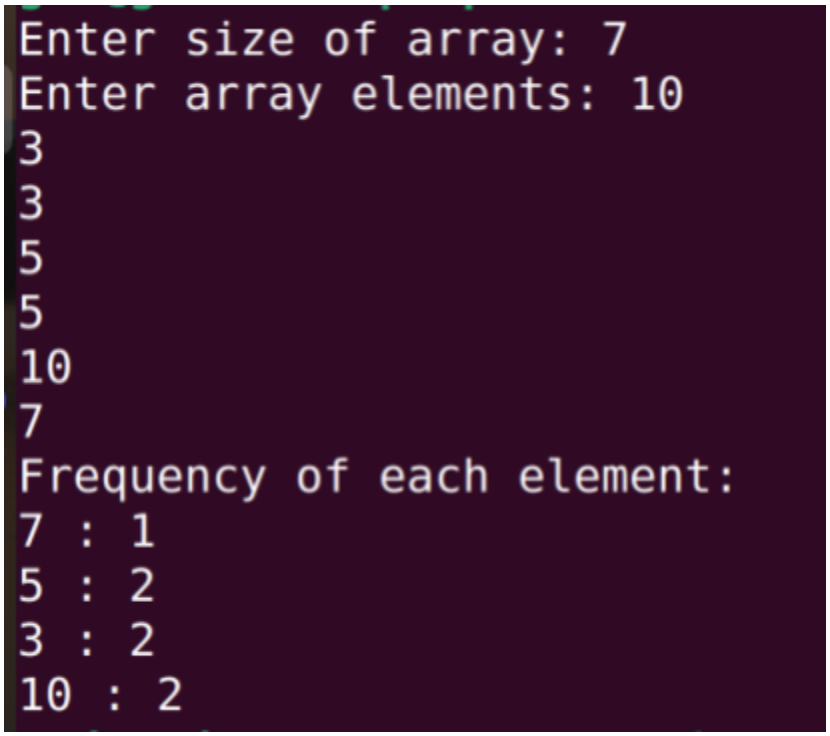
    // Find frequency
    for (int i = 0; i < n; i++) {
        hash[arr[i]]++;
    }

    // Display frequency
    cout << "Frequency of each element:" << endl;

    for (auto x : hash) {
        cout << x.first << " : " << x.second << endl;
    }

    return 0;
}
```

OUTPUT :


Enter size of array: 7
Enter array elements: 10
3
3
5
5
10
7
Frequency of each element:
7 : 1
5 : 2
3 : 2
10 : 2